

UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

ENGENHARIA DE COMPUTAÇÃO

DAVI CELESTINO DOS SANTOS BARBOSA, 23210541

JOÃO VICTOR TENÓRIO VENANCIO, 23211191

OTAVIO JOSHUA COSTA BRANDAO MENEZES, 23211189

LARISSA FERREIRA DIAS DE SOUZA, 23211102

**RELATÓRIO TÉCNICO: SISTEMA DE RECOMENDAÇÃO CINEMATOGRAFICA
COM AGENTES BASEADOS EM LLM**

MACEIÓ - AL

2026

UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

ENGENHARIA DE COMPUTAÇÃO

DAVI CELESTINO DOS SANTOS BARBOSA, 23210541

JOÃO VICTOR TENÓRIO VENANCIO, 23211191

OTAVIO JOSHUA COSTA BRANDAO MENEZES, 23211189

LARISSA FERREIRA DIAS DE SOUZA, 23211102

**RELATÓRIO TÉCNICO: SISTEMA DE RECOMENDAÇÃO CINEMATOGRAFICA
COM AGENTES BASEADOS EM LLM**

Este relatório descreve a arquitetura, o mecanismo de inferência e a avaliação de uma aplicação baseada em Modelos de Linguagem de Grande Escala (LLMs). O sistema atua como um agente conversacional autônomo capaz de elicitar preferências, extrair parâmetros estruturados e realizar Geração Aumentada por Recuperação (RAG) sobre uma base de dados real de filmes.

Orientador (a): Prof. Dr. Evandro de Barros Costa

MACEIÓ - AL

2026

SUMÁRIO

1. DESCRIÇÃO DO DOMÍNIO.....	4
2. ARQUITETURA E REPRESENTAÇÃO DO CONHECIMENTO.....	4
3. MECANISMO DE INFERÊNCIA UTILIZADO.....	5
Fase 1: Gerenciamento de Diálogo (Dialogue Manager).....	5
Fase 2: Extração Estruturada (State Extraction).....	5
Fase 3: Geração Aumentada por Recuperação (RAG - Synthesis).....	5
4. EXEMPLOS DE INTERAÇÃO.....	5
5. ANÁLISE CRÍTICA DAS VANTAGENS E LIMITAÇÕES DA ABORDAGEM.....	6
Vantagens (Pontos Fortes).....	6
Limitações e Trabalhos Futuros.....	6

1. DESCRIÇÃO DO DOMÍNIO

O domínio de aplicação é a Recomendação Personalizada de Filmes. Diferente de sistemas tradicionais baseados em cliques ou filtragem colaborativa, este sistema utiliza o Processamento de Linguagem Natural (PLN) para interagir dinamicamente com o usuário.

O agente tem como objetivo descobrir cinco parâmetros fundamentais do usuário: Gênero cinematográfico, Ano mínimo, Ano máximo, Nota mínima e Nota máxima (0 a 10). Ao invés de depender do conhecimento paramétrico do LLM (que é propenso a alucinações e desatualizações), o domínio é restrito a uma fonte da verdade externa e determinística: o dataset estruturado `movies_metadata.csv`.

2. ARQUITETURA E REPRESENTAÇÃO DO CONHECIMENTO

A aplicação foi construída sob o paradigma de Agentes de Múltiplos Estágios (Multi-stage Agentic Workflow), utilizando o modelo Llama 3.1 (via framework Ollama) rodando localmente. A arquitetura foi dividida em três módulos distintos para garantir separação de responsabilidades (Separation of Concerns):

1. Gestor de Prompts (`prompts.py`): Define a "persona" do agente e as regras de transição de estado. O prompt de sistema instrui o LLM a fazer apenas uma pergunta por vez e a emitir a flag "PRONTO" estritamente quando todas as restrições forem coletadas, agindo como um critério de parada lógico.
2. Motor de Busca e Ferramentas (`busca.py`): Representa as Tools do agente. Utiliza a biblioteca `pandas` para carregar o arquivo CSV. O conhecimento é processado deterministicamente: strings de dicionários são convertidas em listas reais (via `ast.literal_eval`), datas são tipadas, e regras de negócio são aplicadas (ex: `vote_count >= 50` para evitar filmes desconhecidos com notas artificialmente altas).
3. Orquestrador Central (`agente.py`): O script principal que controla o fluxo de dados. Ele gerencia o histórico de mensagens, faz as chamadas à API do Ollama e orquestra a transição entre a fase conversacional, a fase de extração e a fase de síntese final.

3. MECANISMO DE INFERÊNCIA UTILIZADO

O sistema não utiliza o LLM como um mero "gerador de texto", mas sim como um mecanismo de raciocínio dividido em três papéis distintos ao longo do pipeline:

Fase 1: Gerenciamento de Diálogo (Dialogue Manager)

O LLM recebe o histórico da conversa e infere qual informação está faltando com base no System Prompt. Ele gera perguntas naturais para preencher as lacunas do estado (ex: "Qual a nota mínima que você espera?"). Ao perceber que possui os dados, infere a condição de saída e retorna "PRONTO".

Fase 2: Extração Estruturada (State Extraction)

Para transitar da linguagem natural imprecisa para uma chamada de função rígida no Pandas, o orquestrador invoca o LLM em um modo de Saída Estruturada (JSON Mode).

Um schema JSON estrito é passado ao modelo, forçando-o a ler o histórico da conversa e inferir/mapear as intenções do usuário em variáveis de tipagem forte (genero como string, anos como integer, notas como number).

Fase 3: Geração Aumentada por Recuperação (RAG - Synthesis)

Com o JSON em mãos, o script aciona a ferramenta algorítmica `buscar_filmes()`. Os 5 melhores resultados são formatados em uma grande string (contexto) e enviados para uma terceira inferência do LLM.

Nesta última etapa, o LLM recebe um prompt estrito de Grounding ("Não invente filmes. Use somente os filmes enviados"). Ele lê os dados brutos recuperados do CSV e sintetiza uma recomendação humanizada, explicando o porquê de os filmes baterem com os filtros do usuário.

4. EXEMPLOS DE INTERAÇÃO

Abaixo, um fluxo de execução documentado demonstrando a capacidade do agente de guiar o usuário e acionar ferramentas:

- Agente (LLM Fase 1): "Olá! Que tipo de filme (gênero) você gostaria de assistir hoje?"
- Usuário: "Quero ver um filme de Action."

- Agente (LLM Fase 1): "Ótima escolha! A partir de que ano você gostaria que o filme fosse?"
- Usuário: "Dos anos 90 pra cá, até 2010. E que tenha nota maior que 8."
- Agente (LLM Fase 1): "PRONTO"
- (Acontece a transição no backend)
- Extração JSON (LLM Fase 2): {"genero": "Action", "ano_min": 1990, "ano_max": 2010, "nota_min": 8.0, "nota_max": 10.0}
- Busca no Pandas: Filtra os dados brutos e encontra filmes como The Dark Knight e The Matrix.
- Resposta Final (LLM Fase 3): "Com base nas suas preferências, tenho excelentes recomendações de Ação dos anos 90 até 2010 com notas altíssimas: \n1. The Dark Knight (2008) - Nota: 8.3: Uma obra prima que se encaixa perfeitamente no seu filtro de notas altas. \n2. The Matrix (1999) - Nota: 8.1: Um clássico da Ação..."

5. ANÁLISE CRÍTICA DAS VANTAGENS E LIMITAÇÕES DA ABORDAGEM

Vantagens (Pontos Fortes)

1. Mitigação Absoluta de Alucinações: Como a recomendação final exige que o LLM responda estritamente baseado no texto gerado pela função Pandas (lista_filmes), a chance de o agente inventar um filme inexistente ou mentir sobre o ano de lançamento foi praticamente reduzida a zero.
2. Execução Local e Privacidade: A utilização do modelo genérico Llama 3.1 acoplado à engine do Ollama demonstra que fluxos altamente complexos não dependem de APIs pagas (como OpenAI), garantindo total privacidade dos dados do usuário na máquina local.
3. Resiliência e Modularidade: O uso do json schema garante que a ponte entre o raciocínio difuso do LLM e a lógica rígida da programação tradicional (Python/Pandas) nunca quebre por erros de formatação.

Limitações e Trabalhos Futuros

1. Limitações Lexicais: Atualmente, a busca na função buscar_filmes funciona por exatidão de string (genero in generos). Se o usuário digitar "filme de tiro", o LLM tentará usar isso como string, não encontrando o gênero oficial

"Action". Isso poderia ser resolvido utilizando Busca Vetorial e Embeddings em vez de filtros condicionais puros do Pandas.

2. Latência Escalonada: O fluxo ideal necessita de múltiplas chamadas ao modelo (Fase de Diálogo, Extração JSON, Fase de Síntese). Em hardwares com pouca VRAM, gerar inferências sequenciais pode causar lentidão na resposta final ao usuário.
3. Ausência de Ferramentas Nativas (Tool Calling): Em versões futuras, a lógica de duas etapas (descobrir estado -> formatar JSON) poderia ser condensada utilizando a funcionalidade de Function Calling / Tool Use nativa de modelos modernos, onde o LLM invoca a função `buscar_filmes` diretamente por si mesmo.